```
                        ],
                        [
                            5.897759,
                            49.442667
                        ],
                        [
                            5.674052,
                            49.529484
                        ],
                        [
                            5.782417,
                            50.090328
                        ],
                        [
                            6.043073,
                            50.128052
                        ]
                    ]
                ]
            }
        }
    ]
}
```

We're not going to create our own GeoJSON in this chapter, and unless you get into serious GIS, you may never create your own GeoJSON. Instead, you can get by with downloading existing geodata, and either use it without editing it or edit it in a GIS application and export it. In our examples in this chapter, we'll use world.geojson (available at emeeks.github.io/d3ia/world.geojson), a file that consists of the countries of the world in the same simplified, low-resolution representation that you see in listing 7.4.

PROJECTION

Entire books have been written on creating web maps, and an entire book could be written on using D3.js for crafting maps. Because this is only one chapter, I'll gloss over many deep issues. One of these is projection. In GIS, *projection* refers to the process of rendering points on a globe, like the earth, onto a flat plane, like your computer monitor. You can project geographic data in many different ways for representation on your screen, and in this chapter we'll look at a few different methods.

To start, we'll use one of the most common geographic projections, the Mercator projection, which is used in most web maps. It became the de facto standard because it's the projection used by Google Maps. To use the Mercator projection, you have to include an extension of D3, `d3.geo.projection.js`, which you'll want for some of the more interesting work you'll do later in the chapter. By defining a projection, you can take advantage of `d3.geo.path`, which draws geoData onscreen based on your selected projection. After we've defined a projection and have `geo.path()` ready, the entire code in the following listing is all that we need to draw the map shown in figure 7.3.